PROCESS SYNCHRONIZATION SIMULATOR

By

MUHAMMAD UZAIR

(2023-UoK-03680)

SYED SAQLAIN HAIDER

(2023-UoK-03688)

AUSAF AHMED

(2023-UoK-03691)

Session 2023-2027

Department of CS & IT

Faculty of Computing & Engineering

University of Kotli Azad Jammu & Kashmir

UNIVERSITY OF KOTLI AZAD JAMMU & KASHMIR

A proposal for the Project of Operating System in BSCS

Title: PROCESS SYNCHRONIZATION SIMULATOR

Name of Students:	**Muhammad Uzair, Syed Saqlain Haider, Ausaf Ahmad**
Registration Number:	**2023-UoK-03680, 2023-Uok-03688, 2023-UoK-03691**
Session:	**2023-2027**
Date of Submission:	**29-01-2026**
Probable Duration:	**5TH Semester**

Instructor: Mr. Mehtab Mushtaq Signature:_____

CONTENT

1. Project Objective 4

2. Functional Requirements 4

2.1. Process Management 4

2.2. Critical Section Handling 4

2.3. Synchronization Mechanisms 4

2.4. Execution Control 5

2.5. Monitoring & Display 5

..... 5

3. Non-Functional Requirements 5

..... 6

4. Input Requirements 6

5. Output Requirements 7

6. Assumptions & Constraints 8

6.1. Assumptions 8

6.2. Constraints 8

7. References 8

1. Project Objective

The objective of this project is to design and implement a Process Synchronization System that simulates how an operating system controls access to shared resources among multiple concurrent processes.

In real operating systems, multiple processes run at the same time and may need to access shared data such as files, memory, or hardware devices. Without synchronization, this leads to race conditions, data inconsistency, and system errors.

This system demonstrates:

- Why synchronization is necessary
- How critical sections are protected
- How synchronization mechanisms prevent conflicts, such as Java synchronization for threads

The project is for educational purpose, not an actual OS kernel. It focuses on understanding behavior, not controlling hardware.

2. Functional Requirements

The system must provide the following functionalities:

2.1. Process Management

- Allow the user to create multiple processes
- Assign each process an ID and execution time
- Simulate concurrent execution of processes

2.2. Critical Section Handling

- Define a shared resource accessed by multiple processes
- Clearly mark entry section, critical section, and exit section
- Ensure only one process enters the critical section at a time

2.3. Synchronization Mechanisms

- Implement synchronization techniques such as:
    - Mutex Locks
    - Semaphores (Binary and Counting)

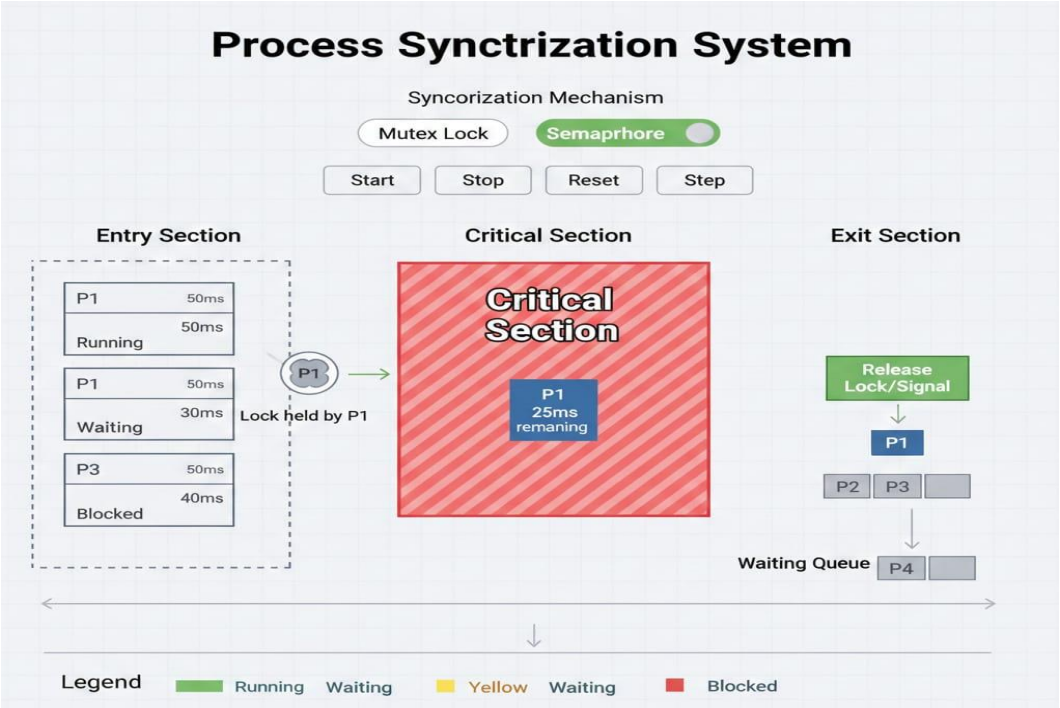
- Allow the user to choose which mechanism to apply

2.4.Execution Control

- Start, stop, reset, and step through the simulation
- Show process states (Running, Waiting, Blocked)

2.5.Monitoring & Display

- Show which process currently holds the lock/semaphore
- Display waiting queue of blocked processes
- Indicate deadlock or starvation if conditions arise (optional)

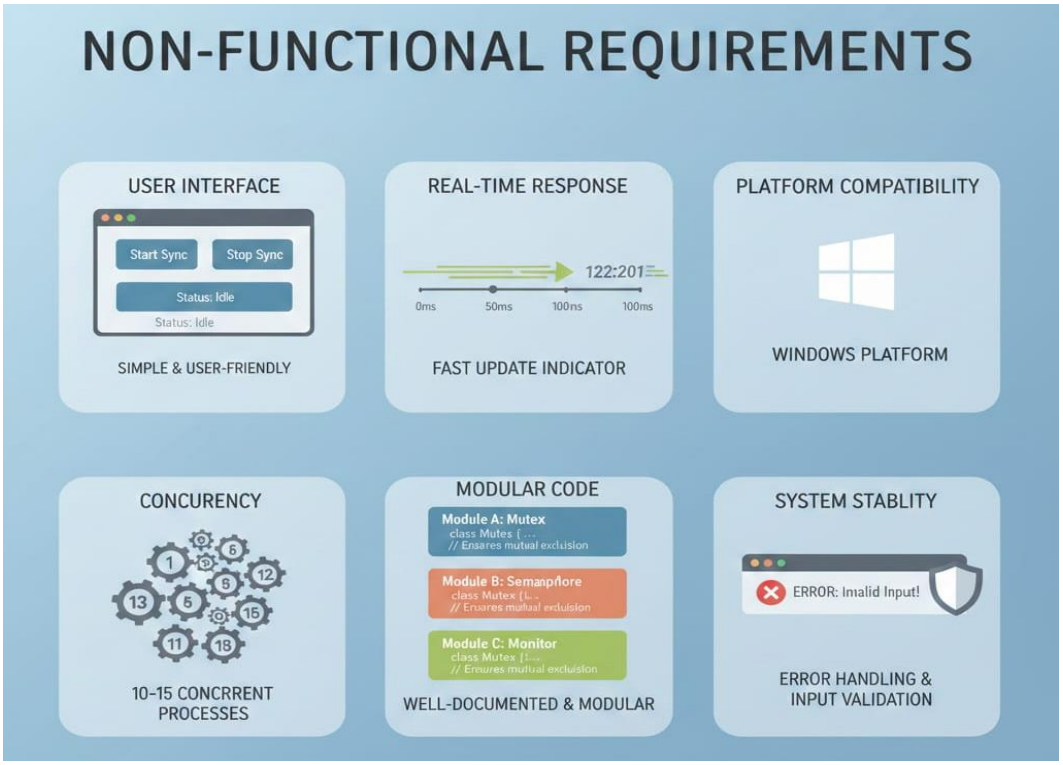


3. Non-Functional Requirements

The system must satisfy the following quality constraints:

- User interface should be simple and understandable
- System should respond in real-time during simulation
- Must run on Windows platforms

- Should support at least 10–15 concurrent processes
- Code should be modular and well-documented
- The simulator should not crash due to invalid input

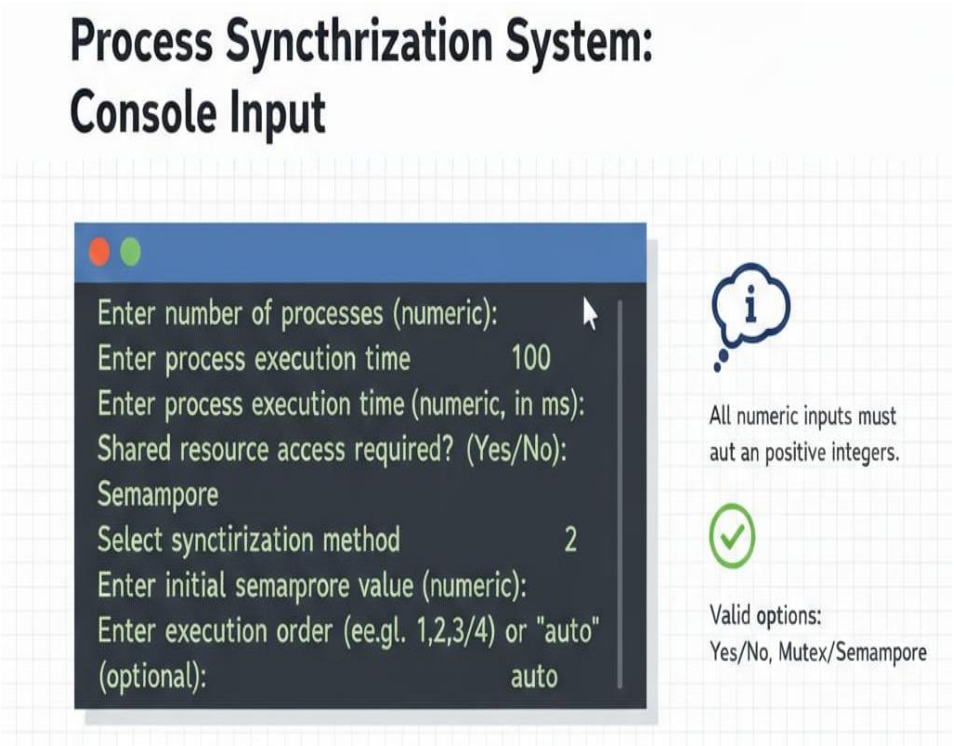


4. Input Requirements

The system requires the following inputs from the user:

- Number of processes
- Process execution time
- Shared resource access requirement (Yes/No)
- Synchronization method selection (Mutex / Semaphore)
- Semaphore value (if applicable)
- Execution order or arrival time (optional)

All inputs must be numeric and valid as Shown in below fig:



5. Output Requirements

The system will display the following outputs:

- Current state of each process (Running, Waiting, Blocked)
- Which process is in the critical section
- Lock/semaphore status (Locked/Unlocked, Value)
- Waiting queue of blocked processes
- Execution log showing order of access
- Final summary of synchronization behavior

Output may be shown using:

- Console-based output
- Simple GUI visualization (optional)

6. Assumptions & Constraints

6.1.Assumptions

- All processes are simulated software processes, not real OS processes.
- Processes are independent, except when accessing the shared resource.
- User provides valid and numeric input values.
- Synchronization rules (mutex/semaphore) are correctly followed during simulation.
- The system focuses only on synchronization behavior, not hardware execution.

6.2.Constraints

- Maximum number of processes is limited i.e. 10-15.
- Simulation does not represent real CPU scheduling
- Deadlock detection is basic or optional
- This is a learning tool, not a production OS component

7. References

1. Tutorialspoint. (n.d.). Process synchronization. In Operating system tutorial. https://www.tutorialspoint.com/operating_system/os_process_synchronization.htm
2. Oracle. (n.d.). Java concurrency and synchronization. Oracle Java Documentation. https://docs.oracle.com/javase/tutorial/essential/concurrency/
3. ChatGPT (Assistance in rewriting and refining content for clarity & structure)
4. Google Gemini (For Illustrations)